



第 14 讲：数据

Stanford CS336：从零开始的语言建模

2026 年春季

课程笔记

Contents

1	数据转换	3
1.1	HTML 到文本的转换	3
1.2	PDF 到文本的转换	4
1.3	基于规则 vs 基于模型的转换	6
1.4	本章小结	6
2	数据过滤	7
2.1	过滤的基本框架	7
2.1.1	生成模型 vs. 判别分类器	7
2.2	过滤实例	8
2.2.1	语言识别	8
2.2.2	数学文本过滤: OpenWebMath	9
2.2.3	质量过滤: GPT-3 与 Phi-1	9
2.2.4	毒性过滤: Jigsaw 评论数据集	9
2.3	过滤阈值与训练规模的权衡	10
2.3.1	直观理解	10
2.3.2	实验证据	10
2.3.3	学生问答中的关键洞见	11
2.4	本章小结	12
3	数据去重	12
3.1	动机与重复类型	12
3.2	精确去重	14
3.3	近似去重: MinHash 与 LSH	14
3.3.1	Jaccard 相似度	14
3.3.2	MinHash 原理	15
3.3.3	局部敏感哈希 (LSH)	16
3.3.4	S 型碰撞概率曲线与相变效应	17
3.3.5	参数 B 与 R 的调优	17
3.4	本章小结	18
4	数据混合	18
4.1	基本问题	19
4.2	数据源规模与训练轮次的约束	19
4.3	基于回归的混合优化	20
4.4	从小规模到大规模的扩展	22
4.5	领域细分与混合粒度	22
4.6	训练中的批次级实现	23
4.7	本章小结	23

5	后训练数据与合成数据	23
5.1	后训练数据概述	24
5.2	数学与科学推理合成数据	25
5.3	代码与智能体合成数据	26
5.3.1	SWE-smith: 从代码仓库自动生成任务	26
5.3.2	Agentless / OpenHands: 智能体轨迹生成	26
5.3.3	SWE-bench: 大规模拉取请求数据集	27
5.4	合成数据的权衡	27
5.5	本章小结	28
6	总结与延伸	28
6.1	核心要点回顾	28
6.2	从数据到能力的因果链条	29
6.3	开放问题与前沿方向	29
6.4	实践建议	29

1 数据转换

原始网络数据并非以纯文本形式直接存在。从互联网抓取的内容通常以 HTML、PDF 或代码仓库等形式存储，需要经过一系列转换才能变为可供语言模型训练的文本格式。本章讨论 HTML 到文本、PDF 到文本的转换过程，以及基于规则与基于模型的转换方法之间的权衡。¹

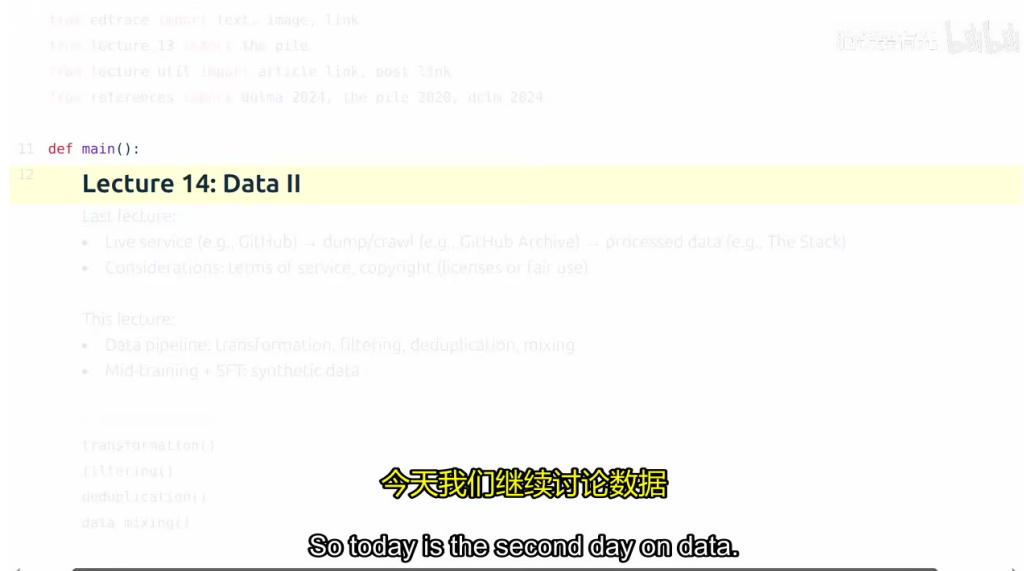


Figure 1: 数据转换流水线概览：从原始网络格式到纯文本的转换过程

2

1.1 HTML 到文本的转换

互联网上的大部分内容以 HTML（HyperText Markup Language，超文本标记语言）编写。当我们从 Common Crawl 等来源获取数据时，得到的并不是可直接用于训练的纯文本，而是包含大量标记、脚本和样式的 HTML 文档。³

背景知识

Common Crawl 是一个公开的网络存档项目，定期抓取互联网上的网页并提供下载。它是构建大规模语言模型训练语料的最主要来源之一。然而，Common Crawl 提供的原始数据是未经处理的 HTML 文件，需要经过复杂的转换和清洗流程。

将 HTML 转换为文本的过程中，大量运用了启发式方法（heuristics）。核心任务是清理导航栏、广告、页眉页脚等非内容元素，提取页面的主要信息。⁴ 但“内容”与“非内容”之间的界限并不总是明确的：有时导航元素有助于理解网页结构，而页眉页脚又可能包含重要信息。因此，何为内容、何为非内容，需要根据具体场景做出判断。⁵

¹ 视频来源时间：00:00:05--00:01:17

³ 视频来源时间：00:01:18--00:01:50

⁴ 视频来源时间：00:01:51--00:02:01

⁵ 视频来源时间：00:02:02--00:02:32

常见误区

内容边界模糊性：在 HTML 转换中，过度激进的清理可能误删有价值的信息，而过于保守则可能保留大量噪声。例如，导航元素通常被视为非内容，但它们有时能帮助模型理解网页结构；页眉页脚通常被删除，但某些情况下可能包含版权声明或重要元数据。这种权衡没有统一标准，取决于下游任务的需求。

网页中的图片和表格处理尤为棘手。本质上，HTML 转换涉及将层次化的 DOM (Document Object Model, 文档对象模型) 树结构进行线性化 (linearization) ——即将二维的、可能具有可视化渲染效果的结构展平为一维的标记序列。⁶

表格处理是线性化中的典型难题：

- **简单表格：**可以用 Markdown 格式渲染，保留行列关系；
- **嵌套表格：**结构复杂，线性化后难以保持语义，处理起来极具挑战性。⁷

在某些时刻，面对过于复杂的嵌套结构，我们不得不在“放弃该内容”与“近似表示”之间做出权衡。

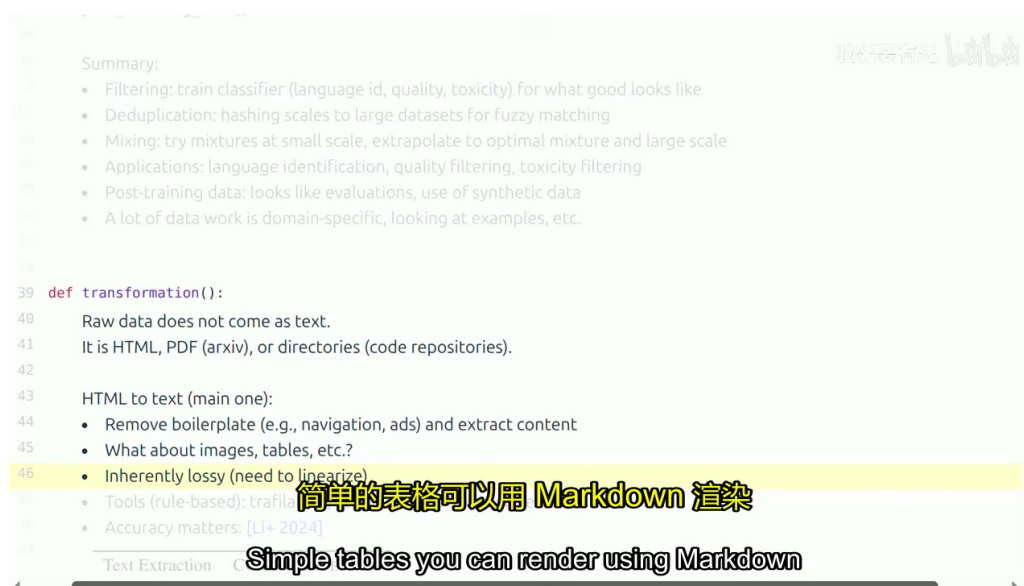


Figure 2: HTML 文档的 DOM 树结构与线性化挑战

8

1.2 PDF 到文本的转换

除了 HTML，互联网上还存在大量 PDF (Portable Document Format, 便携式文档格式) 文件。HuggingFace 发布了名为 `fineweb-pdf` 的数据集，专门用于研究 PDF 的文本提取问题。⁹

PDF 与 HTML 在设计理念上存在根本差异：

- **HTML** 是语义导向的，标签如 `<h1>`、`<p>` 明确传达了内容的结构意义；

⁶视频来源时间：00:02:33--00:02:54

⁷视频来源时间：00:02:55--00:03:08

⁹视频来源时间：00:04:19--00:04:27

- **PDF 是版式导向的**，从设计上更注重页面布局和视觉效果，而非语义结构。¹⁰

因此，将 PDF 转换为文本时，大量布局信息会丢失，原有的语义结构不一定能被保留。此外，PDF 的文本提取还面临以下挑战：

1. **原始字节 vs 渲染文本**：直接用文本编辑器打开 PDF 文件，看到的不是人类可读的文本，而是经过编码的二进制数据，需要专门的渲染引擎进行解析。¹¹
2. **扫描件 OCR**：有些 PDF 本质上是图像（扫描件），需要使用视觉语言模型（Vision Language Model, VLM）运行光学字符识别（Optical Character Recognition, OCR），这比处理纯文本要昂贵得多。¹²
3. **重新抓取问题**：Common Crawl 中的许多 PDF 由于文件体积大而被截断，因此需要重新抓取原始文件，这涉及许多工程细节。¹³
4. **类型识别**：有时我们只有 URL，在获取前可能不知道它是否为 PDF（尤其是没有扩展名时）。¹⁴

核心要点

PDF 的质量悖论：PDF 在整个互联网中只占很小一部分，但它们通常具有更高的信息密度和质量。因为制作 PDF 一般都有特定目的（如学术论文、技术文档、报告），这意味着相比普通网页，PDF 中可能包含更重要的内容。因此，尽管 PDF 处理成本高，其投入产出比往往值得。^a

^a视频来源时间：00:05:46--00:06:14

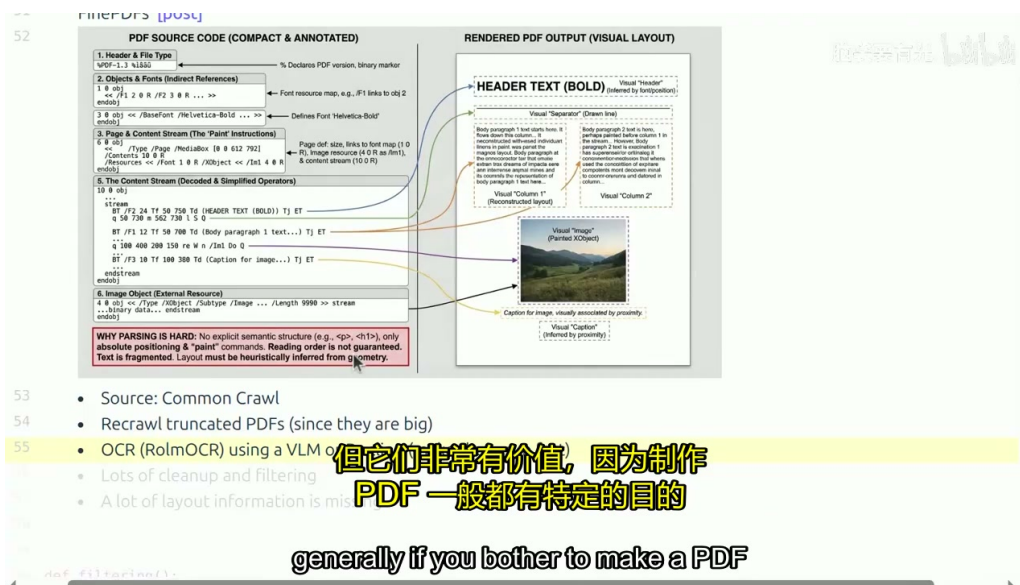


Figure 3: PDF 文本提取中的版式导向与语义导向之争

15

¹⁰视频来源时间：00:06:24--00:06:35

¹¹视频来源时间：00:04:28--00:04:39

¹²视频来源时间：00:05:25--00:05:45

¹³视频来源时间：00:05:08--00:05:18

¹⁴视频来源时间：00:04:55--00:05:03

1.3 基于规则 vs 基于模型的转换

HTML 到文本的转换通常基于规则（rule-based）进行。这种方法的优势在于速度快、实现简单、不需要太多智能——只需定义一系列启发式规则（如删除特定标签、提取特定区域），往往就能取得不错的效果。¹⁶

然而，基于规则的处理必然存在一定的失败率。观察转换后的数据，总能发现一些瑕疵——可能是广告未被完全清除，也可能是主要内容被误删。¹⁷

近年来，基于模型的介入（model-based transformation）成为一种潜在方向。理论上，使用语言模型或专门的文档理解模型可以更智能地识别内容边界、处理复杂表格、甚至理解页面布局。但这些方法需要更快速、更智能地运行，才能在万亿级 token 的数据处理中具备实用性。¹⁸

常见误区

工具选择影响下游性能：正如课程前面展示的，数据转换的准确性对最终模型性能至关重要。不同的转换工具在扩展的 DCLM（DataComp for Language Models）评估上表现差异显著。例如，使用 `trafilatura` 等工具时，其在 DCLM 评估上优于其他替代方案。选择转换工具不是中性的工程决策，而是直接影响模型质量的关键环节。^a

^a视频来源时间：00:03:57--00:04:18

Table 1: 基于规则与基于模型的转换方法对比

维度	基于规则	基于模型
速度	快（线性时间）	慢（需模型推理）
成本	低（CPU 即可）	高（需 GPU/TPU）
准确率	中等（存在失败率）	高（理解能力强）
可扩展性	优秀	受限于推理吞吐量
适用场景	大规模预训练数据处理	小规模高质量数据提取

当前工业界的实践仍以基于规则的方法为主流，模型介入主要用于特定的高质量数据提取场景。随着模型推理成本的下降，两者之间的边界可能会逐渐模糊。

1.4 本章小结

数据转换是将原始网络数据（HTML、PDF 等）转化为可供语言模型训练的纯文本格式的第一步，也是后续过滤、去重与混合操作的前提。本章的核心要点如下：

1. **HTML 线性化**是数据转换的主要任务，涉及启发式清理、内容边界判断和表格等复杂结构的展平处理。内容与非内容的界限并不明确，需要根据场景权衡。
2. **PDF 转换**面临语义结构丢失、扫描件 OCR、文件截断和类型识别等挑战。尽管 PDF 在互联网中占比小，但其信息密度高、质量通常优于普通网页，值得投入处理成本。

¹⁶视频来源时间：00:03:10--00:03:33

¹⁷视频来源时间：00:03:45--00:03:53

¹⁸视频来源时间：00:03:34--00:03:44

3. **基于规则的方法**因速度快、成本低而占据主流，但存在一定失败率；**基于模型的方法**理论上更智能，但推理成本限制了其在大规模数据处理中的应用。工具的选择直接影响下游模型性能。

转换完成后，文本数据已经形成，但距离可用于训练的高质量语料还有相当距离。接下来的步骤是数据过滤——从海量原始文本中筛选出符合质量标准的子集。

2 数据过滤

在将原始 HTML 或 PDF 文档转换为纯文本之后，我们得到的数据仍然远未达到可直接用于训练语言模型的标准。网络上充斥着大量低质量、无关甚至有害的内容，因此**数据过滤**（Data Filtering）成为构建高质量预训练数据集的核心环节之一。本节将介绍过滤的通用框架、典型实例，以及过滤阈值与训练规模之间的关键权衡。¹⁹

2.1 过滤的基本框架

过滤问题的本质可以概括为：给定一小部分目标高质量数据 T （Target data）和海量原始数据 R （Raw data），我们的目标是找出 R 中与 T 相似的子集。这一框架几乎适用于所有类型的过滤任务——无论是语言识别、质量筛选还是毒性过滤。²⁰

核心要点

过滤的通用架构：目标数据 T + 原始数据 R $\rightarrow$ 评分函数 $\rightarrow$ 阈值 $\rightarrow$ 保留子集。几乎所有过滤方法都遵循这一模式，区别仅在于如何定义“目标”以及如何计算评分。

过滤的动机是多方面的。如果你正在训练英语或德语语言模型，首先需要识别并移除非目标语言的内容；如果你追求高质量输出，就需要过滤掉垃圾信息，保留百科全书式的内容；此外，互联网上存在大量有害内容，**毒性过滤**（Toxicity Filtering）也日益受到重视。²¹

从算法角度，过滤需要满足两个关键约束：一是**泛化能力**——不能仅局限于已收集的目标数据 T ，而要能从 R 中推广到更多相似样本；二是**效率**——面对互联网上多达 100 万亿个标记的数据量，过滤算法必须足够快速。实践中，过滤后通常仅保留个位数比例的数据。²²

2.1.1 生成模型 vs. 判别分类器

基于上述框架，通常有两种构建评分函数的思路：

1. **生成模型**（Generative Model）：直接对目标数据 T 建模，例如训练一个五元语法模型（5-gram model），然后用该模型评估新文档的似然度。这种方法直观，但泛化能力有限。
2. **判别分类器**（Discriminative Classifier）：将 T 视为正样本，从 R 中采样一部分作为负样本，训练一个二分类器。对于目标数据中的样本预测正标签，对于原始数据中不在目标集合内的样本预测负标签。²³

¹⁹视频来源时间：00:06:14--00:06:56

²⁰视频来源时间：00:06:56--00:07:38

²¹视频来源时间：00:07:38--00:08:18

²²视频来源时间：00:08:18--00:08:54

²³视频来源时间：00:08:54--00:10:03

如今，判别分类器更为常见。其中，**FastText** 因其速度极快且通常只需一个线性分类器，成为业界广泛采用的工具。训练完成后，对每个新文档进行评分，并根据预设的质量阈值决定是否保留。²⁴

背景知识

FastText 是 Meta 开源的快速文本分类与词嵌入工具，常用于语言识别和质量分类。它的核心优势在于：基于词袋特征（bag-of-words）训练线性分类器，推理速度极快，能够轻松处理万亿级标记的网页数据。许多大规模语言模型的数据流水线都将其作为标准组件。

值得注意的是，基于模型的过滤方法已成为主流。几年前，许多数据集为了避免引入过大偏差而避免使用模型过滤；但如今，除非计算资源极其充裕，否则大家都会进行一定程度的基于模型的过滤——否则就是对低质量内容浪费宝贵的计算资源。²⁵

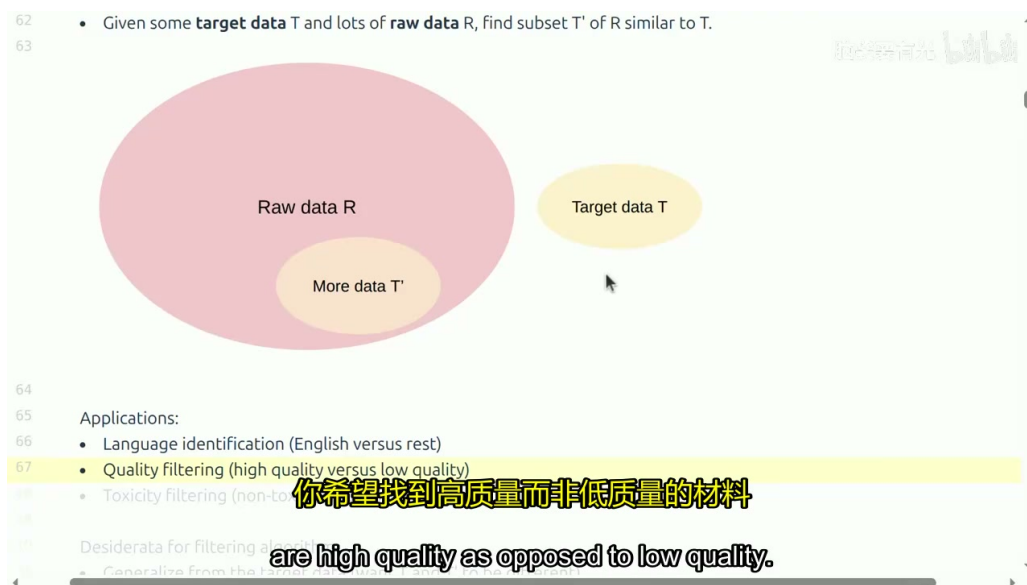


Figure 4: 数据过滤的通用框架：从目标数据到评分阈值

26

2.2 过滤实例

2.2.1 语言识别

语言识别是最基础的过滤任务之一：给定一段文本，判断它属于哪种语言。Meta 训练了一套快速的文本语言识别模型，支持 176 种不同语言，可直接拿来使用。该模型在维基百科、翻译网站等多语言资源上训练，对于常见语言（如区分西班牙语和日语）只需查看几个词即可做出判断。²⁷

当然，语言识别并非完全解决的问题。**代码切换**（code switching，同一文本中混用多种语言）、方言差异以及文本的复杂性都会带来挑战。但对于训练语言模型而言，语言识别通常不是瓶颈——一个简单的分类器配合适当的阈值即可满足大部分需求。²⁸

²⁴ 视频来源时间：00:10:03--00:10:42

²⁵ 视频来源时间：00:10:42--00:11:26

²⁷ 视频来源时间：00:11:26--00:12:28

²⁸ 视频来源时间：00:12:28--00:12:50

2.2.2 数学文本过滤：OpenWebMath

如果你想让模型在数学方面表现更出色，就需要专门收集数学数据。OpenWebMath（2023 年发表）正是为了构建大规模数学文本库而设计的多阶段过滤流程。²⁹

该流程包含以下步骤：

1. **规则过滤**：首先用规则筛选是否包含 $\text{IAT}_\text{E}^\text{X}$ 命令，快速排除明显非数学的文档。
2. **困惑度过滤**（Perplexity Filtering）：使用在已知数学证明数据集上训练的生成式模型（如 CANALLION）评估文档困惑度，若未超出阈值则保留。
3. **分类器过滤**：训练 FastText 分类器判断是否为数学文本。
4. **分层阈值**：对包含 $\text{IAT}_\text{E}^\text{X}$ 的文档采用较低门槛（标准更高），对不包含 $\text{IAT}_\text{E}^\text{X}$ 的文档采用较高门槛。³⁰

通过这一精细流程，OpenWebMath 收集了约 150 亿个数学标记。论文指出，这种更精准的数据收集方式产生的模型，在数学能力上超越了使用 20 倍未经过滤数据训练的模型——这充分说明了**精准过滤比盲目堆量更有效**。³¹

2.2.3 质量过滤：GPT-3 与 Phi-1

GPT-3 的质量过滤策略可以完美嵌入我们的通用框架：正样本包括维基百科、网页文本以及一些书籍；负样本则多来自网络。他们训练了一个线性分类器，评分高的文档被保留。有趣的是，第一篇 Llama 论文中，正样本是被维基百科引用的页面，而非维基百科文章本身——这体现了“目标数据”定义的灵活性。³²

微软的 Phi-1 则展示了另一种思路。原始数据全部是代码，他们定义了一个提示词（prompt）来确定“教育价值”，然后利用 GPT-4 对原始数据中的 10 万个子集进行分类。被判定为正例的样本成为目标 T ，随后训练一个更简单的分类器（随机森林或 FastText）来筛选全部数据。结果表明，使用这组过滤后的数据比直接使用原始数据效果更好，且能在更少的训练步骤中达到更高性能。³³

2.2.4 毒性过滤：Jigsaw 评论数据集

Jigsaw 评论数据集是毒性过滤的经典案例。该项目旨在帮助人们在网上进行更有效的讨论，数据来源于维基百科讨论页——尤其是争议性强的页面。每条评论都标注了是否含有毒性。基于这些标注，可以定义正样本（无毒）和负样本（有毒），进而训练分类器用于过滤。³⁴

常见误区

质量标准没有统一答案。过滤应被视为一种工具，你可以按自己的方式定义“质量”。如果你希望模型擅长数学，就定义数学质量；如果你希望模型避免有害输出，就定义毒性标准。关键在于：目标数据 T 的选择直接决定了过滤后数据的特性，进而影响模型的能力

²⁹视频来源时间：00:12:50--00:13:14

³⁰视频来源时间：00:13:14--00:13:55

³¹视频来源时间：00:13:55--00:14:27

³²视频来源时间：00:14:27--00:15:17

³³视频来源时间：00:15:17--00:16:32

³⁴视频来源时间：00:16:32--00:17:15

分布。^a

^a视频来源时间：00:14:16--00:14:27

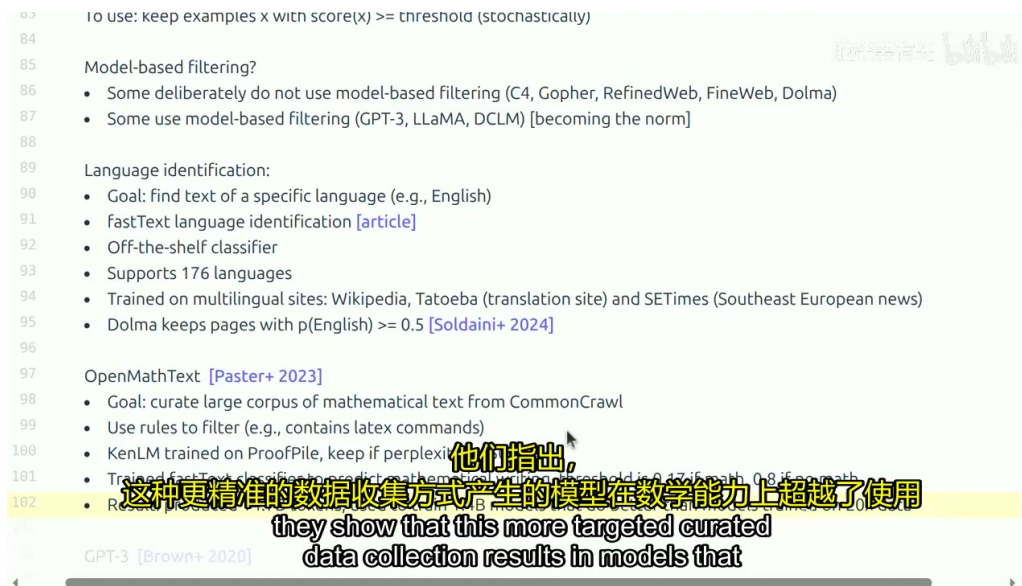


Figure 5: 不同项目的过滤策略实例

35

2.3 过滤阈值与训练规模的权衡

现在你已经掌握了过滤的工具和灵感：明确想要的数据类型，构建分类器，然后过滤网络爬取数据。但这里有一个关键细节：你想要的过滤强度取决于你计划训练的标记数量。并不存在 universally optimal 的阈值。³⁶

2.3.1 直观理解

直观来看：

- **训练时间短**（标记量少）：你需要更高质量的数据，因为模型没有足够的机会从低质量数据中提取有用信号。
- **训练时间长**（标记量多）：你可以接受质量稍低的数据，因为模型有足够的时间学习；但你也希望有更多高质量数据——只是现实中数据来源有限。³⁷

2.3.2 实验证据

讲师展示了一个初步实验（Michael Ryan 的研究），使用一个 1.5 亿/7 亿参数的模型，在一个小的通用爬取样本池（如 100 亿标记）上进行长时间持续训练。³⁸

³⁶视频来源时间：00:17:15--00:17:54

³⁷视频来源时间：00:17:54--00:18:28

³⁸视频来源时间：00:18:28--00:18:53

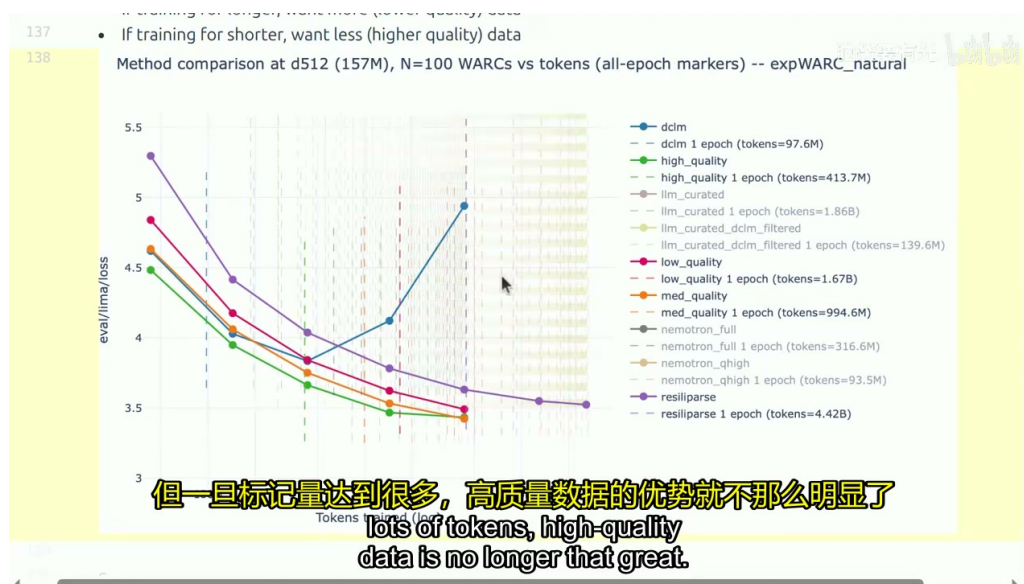


Figure 6: 过滤阈值与训练规模的权衡关系

39

实验结果揭示了关键趋势：

- **高质量数据**（如 DCLM 过滤结果）：初始损失较低，随 epoch 增加持续下降，但重复多轮后会出现**过拟合**（损失反而上升）。
- **低质量数据**（几乎无过滤）：初始损失高得多，但在大量训练后也能逐渐下降，只是学习效率低。
- **核心发现**：高质量数据在短训练中优势明显；但当标记量达到很多时，高质量数据的相对优势就不那么明显了。⁴⁰

常见误区

过拟合风险：使用高质量小数据集进行长时间训练时，必须警惕过拟合。图中显示，高质量数据在第二个 epoch 后损失继续下降，但到某个点后开始过拟合。此时即使继续训练，效果也比用低质量数据训练更长时间还要差。因此，**高质量数据不应被过度重复使用**。^a

^a视频来源时间：00:19:58--00:20:16

2.3.3 学生问答中的关键洞见

讲座中的学生问答进一步澄清了几个要点：

1. **置信区间**：理想情况下应多次运行实验并计算置信区间，但预训练实验成本极高，论文中很少这样做。不过讲师指出，至少在预训练阶段，结果通常是稳定的。⁴¹
2. **边际收益递减**：每个数据集最终都会出现边际收益递减的现象——这是有限的。但高质量数据的递减点会出现在更后面。如果能获取更高质量的数据并训练更长时间，递减仍然会发生，只是位置更靠后。⁴²

⁴⁰视频来源时间：00:18:53--00:19:58

⁴¹视频来源时间：00:20:39--00:21:10

⁴²视频来源时间：00:21:10--00:21:54

核心要点

过滤阈值没有最优解。0.9 的阈值不一定比 0.8 更好——具体取决于你的训练规模。短训练需要更严格的过滤；长训练可以接受更宽松的阈值。这是数据过滤与训练预算之间的核心权衡。^a

^a视频来源时间：00:17:44--00:17:54

2.4 本章小结

过滤是构建优质语言模型的关键步骤，尤其在计算资源受限时。本章的核心要点如下：

1. **通用框架：**所有过滤都可抽象为“目标数据 T + 原始数据 $R \rightarrow$ 评分函数 $\rightarrow$ 阈值”的流水线。判别分类器（尤其是 FastText 线性分类器）因其速度和可扩展性成为主流选择。
2. **质量定义是主观的：**没有统一的质量标准。OpenWebMath 追求数学能力，GPT-3 追求百科质量，Phi-1 追求教育价值——目标数据的选择决定了模型的能力偏向。
3. **阈值与规模相关：**高质量数据在短训练中优势显著，但长训练下低质量数据的边际效用不可忽视。同时必须警惕小高质量数据集上的过拟合风险。
4. **实用策略：**如果你有一个特别喜欢的数据集，可以训练分类器来找到更多类似数据；或者先用强模型（如 GPT-4）对少量数据打标签，再用轻量级分类器（如 FastText）扩展到整个数据集——这种方法既经济又高效。⁴³

过滤完成后，我们得到了自认为高质量的数据集。但数据中仍然可能存在重复项——这正是下一章**数据去重**将要解决的问题。⁴⁴

3 数据去重

经过数据转换与过滤之后，我们得到了一批被认为是高质量的文本数据。然而，这些数据中仍然可能存在大量重复内容。本章将阐述去重的动机与类型，介绍精确去重的线性时间方法，并深入讲解近似去重的核心算法——MinHash 与局部敏感哈希（LSH），包括其数学原理、参数调优与工程实践。

3.1 动机与重复类型

数据中的重复现象极为普遍，大致可分为两类：精确重复（exact duplicate）与近似重复（near duplicate）。⁴⁵

精确重复通常出现在以下场景：

- **镜像站点：**某些网站的存在就是为了复制内容，网络爬虫往往无法分辨它们是否相同，导致抓取到完全一致的页面。
- **代码仓库克隆：**当用户克隆一个仓库时，即使做了少量修改，仓库中绝大部分内容仍然是完全相同的。

⁴³视频来源时间：00:21:54--00:22:54

⁴⁴视频来源时间：00:22:54--00:23:17

⁴⁵视频来源时间：00:22:54--00:23:18

近似重复则更为隐蔽。它们既不是镜像，也不是派生版本，但文本内容基本相同，仅存在轻微差异。常见来源包括：

- **许可证模板**：例如 MIT 许可证可能出现在成千上万个代码仓库中，内容几乎完全一致，仅在复制过程中偶有打字错误。
- **页眉页脚**：许多网站共享相同的页眉和页脚模板。
- **排版差异**：同一篇文章的不同版本可能存在细微的排版差异，例如一个版本有逗号而另一个没有。
- **模板填充**：某些低质量内容看起来像广告，有人套用模板后仅将地名从”加拿大”替换为”美国”。⁴⁶

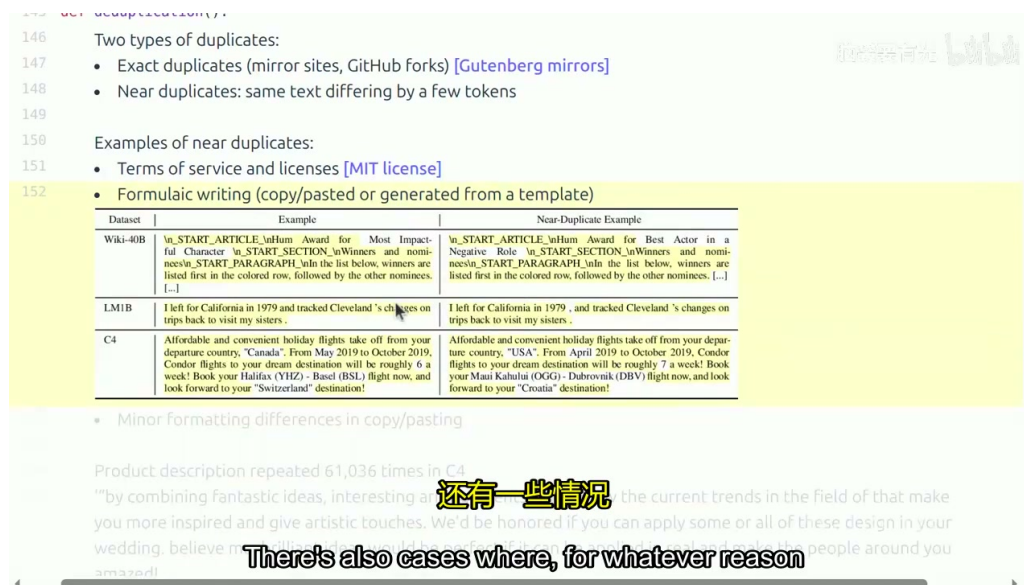


Figure 7: 精确重复与近似重复的典型来源

47

一个极端案例是，审计发现 C4 数据集中某条防毒面具的产品描述出现了 61,000 次之多。如果不对这类重复进行处理，模型将在大量高度相似甚至完全相同的内容上浪费宝贵的 GPU 计算资源。⁴⁸

核心要点

去重至少带来三方面收益：

1. **提高训练效率**：移除重复项后数据集规模减小，但信息并未丢失，从而节省存储与计算成本。
2. **避免记忆问题**：大量重复的版权内容可能导致模型在训练时”记住”这些内容，增加版权与隐私风险。
3. **防止过拟合**：重复数据会人为地提高某些样本的采样频率，导致模型在这些模式上

⁴⁶ 视频来源时间：00:23:18--00:25:28

⁴⁸ 视频来源时间：00:25:28--00:26:17

过拟合。

与去重密切相关的另一个问题是**数据净化**（data decontamination），即确保测试集不包含在训练数据中。这是评估结果可信度的基本前提。⁴⁹

去重还涉及一系列设计选择：去重的粒度是句子、段落还是整篇文档？匹配标准是精确匹配还是近似匹配？一旦发现重复，是移除所有实例还是仅保留一个？本章将重点回答这些问题。⁵⁰

3.2 精确去重

精确去重在概念上非常简单：给定一个字符串，计算其哈希值，检查是否已有完全相同的哈希值存在，若存在则判定为重复，仅保留其中一个副本。⁵¹

背景知识

哈希函数（hash function）将任意长度的输入（如字符串）映射为固定长度的输出（哈希值）。哈希冲突（collision）指两个不同输入映射到同一哈希值的情况。密码学哈希（如 SHA-256）追求抗碰撞性，而哈希表使用的快速哈希函数（如 MurmurHash）则允许可控的冲突概率。去重算法采用的是后者——我们甚至希望冲突以某种可控方式发生，因为冲突概率将与文档相似度直接关联。^a

^a视频来源时间：00:28:47--00:29:31

对于大规模数据集，精确去重通常采用 MapReduce 风格的并行处理：将文档分发到多个节点计算哈希，再通过全局哈希表检测重复。这种方法的时间复杂度为 $O(N)$ ，非常适合扩展。⁵²

一个经典的精确去重案例来自 T5 论文中的 C4 数据集。他们对 Common Crawl 数据进行了精确去重，但操作对象并非整篇文档，而是**三句话的片段**（three-sentence span）。一旦发现两个文档共享相同的三句话片段，便删除除一个之外的所有副本。⁵³

需要注意的是，这种基于片段的精确去重存在争议：从文档中抽取三个句子进行匹配并删除，会破坏文档的连贯性。尽管如此，这一策略在当时被广泛采用，并有效移除了大量重复内容。⁵⁴

3.3 近似去重：MinHash 与 LSH

精确去重对“杂乱”的网络数据来说还不够，因为网页内容经常出现高度相似但并非完全一致的情况。为此，我们需要一种能够衡量**近似相似度**的算法。⁵⁵

3.3.1 Jaccard 相似度

首先定义衡量两个集合相似程度的标准——**Jaccard 相似度**（Jaccard Similarity）。对于两个集合 A 和 B ，其 Jaccard 相似度定义为：

⁴⁹视频来源时间：00:26:17--00:27:17

⁵⁰视频来源时间：00:27:17--00:27:54

⁵¹视频来源时间：00:29:31--00:29:53

⁵²视频来源时间：00:30:00--00:30:29

⁵³视频来源时间：00:30:29--00:31:08

⁵⁴视频来源时间：00:31:08--00:31:26

⁵⁵视频来源时间：00:30:10--00:32:27

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

其中 $|A \cap B|$ 表示交集大小, $|A \cup B|$ 表示并集大小。 $J(A, B)$ 的取值范围为 $[0, 1]$: 0 表示两集合不相交, 1 表示完全相同。⁵⁶

例如, 设 $A = \{1, 2, 3, 4\}$, $B = \{1, 2, 3, 5\}$, 则交集为 $\{1, 2, 3\}$ (大小为 3), 并集为 $\{1, 2, 3, 4, 5\}$ (大小为 5), 因此 $J(A, B) = 3/5 = 0.6$ 。⁵⁷

在实际应用中, 我们将一篇文档视为一组 n -gram (例如 5-gram 的集合), 然后通过比较两个文档的 n -gram 集合的 Jaccard 相似度来判断它们是否近似重复。若相似度超过某个阈值 (如 0.99), 则认为二者近似重复。⁵⁸

3.3.2 MinHash 原理

直接计算两个大规模文档集合的 Jaccard 相似度需要 $O(|A| + |B|)$ 的时间, 若要在 N 个文档之间两两比较, 则总复杂度为 $O(N^2)$, 这对于网络规模的数据集是不可接受的。MinHash 算法通过随机哈希巧妙地将比较复杂度降至线性。⁵⁹

核心要点

MinHash 核心性质: 对于集合 A 和 B , 若使用一个随机哈希函数 h , 则

$$\Pr\left[\min_{x \in A} h(x) = \min_{x \in B} h(x)\right] = J(A, B)$$

即两个集合的 MinHash 值相等的概率, 恰好等于它们的 Jaccard 相似度。这一性质使得我们可以通过哈希碰撞来估计相似度, 从而避免显式计算交集与并集。

算法步骤如下:

1. 将文档表示为 n -gram 的集合 (如所有 5-gram)。
2. 选取一个随机哈希函数 h (由随机种子确定)。
3. 对集合中每个元素计算哈希值, 取其中的最小值作为该文档的 **MinHash 签名**。
4. 两个文档的 MinHash 签名相同 (发生碰撞) 的概率即为 $J(A, B)$ 。

直观上, 随机哈希函数对元素产生了一个随机排列。在所有元素中, 成为“第一个” (即哈希值最小) 的元素, 恰好来自交集的概率就是 $J(A, B)$ 。若该元素同时属于 A 和 B , 则两集合的 MinHash 值必然相等; 若来自 A 或 B 的独有部分, 则最小值不同。⁶⁰

实践中, 我们生成 K 个独立的哈希函数 (通过 K 个不同种子), 得到长度为 K 的 MinHash 签名向量。通过比较两个文档签名向量的匹配比例, 即可估计 Jaccard 相似度。⁶¹

⁵⁶视频来源时间: 00:31:35--00:32:20

⁵⁷视频来源时间: 00:31:50--00:32:08

⁵⁸视频来源时间: 00:32:20--00:32:34

⁵⁹视频来源时间: 00:32:34--00:33:17

⁶⁰视频来源时间: 00:33:17--00:36:29

⁶¹视频来源时间: 00:36:29--00:37:27

3.3.3 局部敏感哈希 (LSH)

MinHash 虽然将单次比较的时间降至常数，但碰撞概率直接等于 Jaccard 相似度 J ，这意味着：

- 即使 $J = 0.8$ （高度相似），单次碰撞概率也只有 0.8，方差很大。
- 即使 $J = 0.5$ （中等相似），仍有 0.5 的碰撞概率，导致大量误报。

我们的目标是：当 J 超过某个阈值（如 0.9）时，碰撞概率应接近 1；当 J 低于阈值时，碰撞概率应接近 0。**局部敏感哈希**（Locality Sensitive Hashing, LSH）通过 AND-OR 结构实现了这一**概率放大**。⁶²

LSH 的 AND-OR 波段结构：

1. 将 K 个独立的 MinHash 函数分成 B 个**波段**（band），每个波段包含 R 个哈希函数，满足 $K = B \times R$ 。
2. 对于每个波段，若该波段内所有 R 个哈希值都相同，则称该波段**匹配**。
3. 若两个文档在**至少一个波段**上匹配，则判定它们为候选重复对。

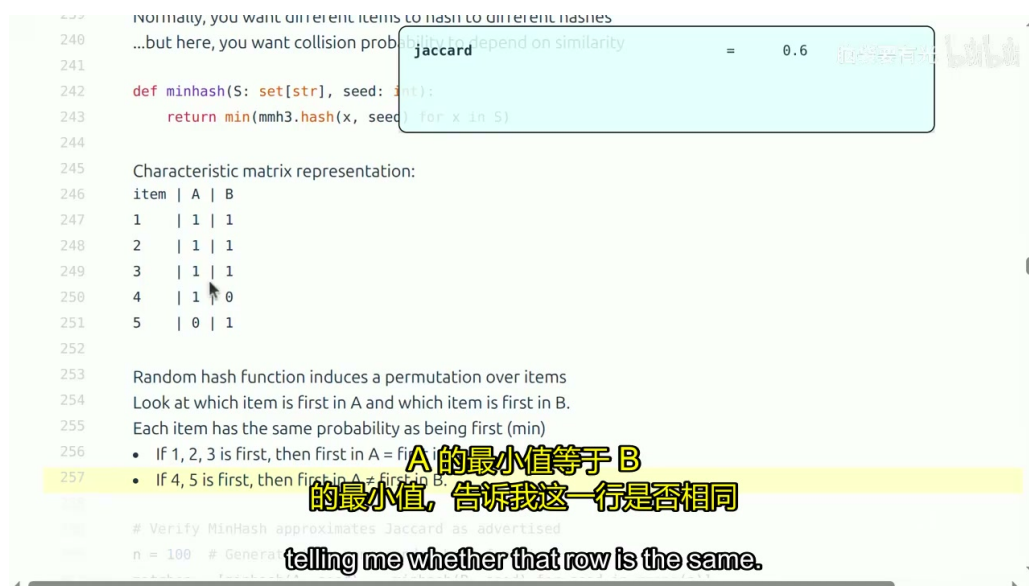


Figure 8: MinHash 签名生成与 LSH 波段结构示意图

63

碰撞概率推导： 设两文档的 Jaccard 相似度为 J 。

- 单个哈希函数碰撞的概率为 J 。
- 单个波段内所有 R 个哈希函数都碰撞的概率为 J^R （AND 结构降低低相似度文档的碰撞概率）。
- 单个波段不匹配的概率为 $1 - J^R$ 。
- 所有 B 个波段都不匹配的概率为 $(1 - J^R)^B$ 。

⁶²视频来源时间：00:37:27--00:39:11

- 至少一个波段匹配的概率（即总体碰撞概率）为：

$$P(\text{collision}) = 1 - (1 - J^R)^B$$

64

3.3.4 S 型碰撞概率曲线与相变效应

将 $P(\text{collision}) = 1 - (1 - J^R)^B$ 按 J 绘制，会得到一条 **S 型曲线** (sigmoid-like curve)：

- 当 $J = 0$ 时， $P = 0$ ；当 $J = 1$ 时， $P = 1$ 。
- 在阈值附近，曲线发生急剧的**相变** (phase transition)：低于阈值的 J 对应的 P 迅速趋近于 0，高于阈值的 J 对应的 P 迅速趋近于 1。

这一 S 型曲线正是我们期望的性质：它能在相似度阈值附近形成清晰的”分水岭”，将高相似度文档与低相似度文档有效区分开。⁶⁵

3.3.5 参数 B 与 R 的调优

参数 B （波段数）和 R （每段哈希数）共同决定了 S 型曲线的形状和位置：

- **增加 R** （每段更多哈希函数）：曲线向右移动，阈值变得更加尖锐，匹配难度增加。因为 J^R 随 R 指数衰减，低相似度文档的碰撞概率被进一步压低。例如，当 $J = 0.9$ 、 $R = 10$ 时， $J^R \approx 0.35$ ；而当 R 增大时，该值迅速下降。⁶⁶
- **增加 B** （更多波段）：曲线向左移动，匹配变得更容易。因为有更多次”尝试”匹配的机会，高相似度文档的碰撞概率被放大。例如，当 B 从 10 增加到更大值时， $J = 0.9$ 对应的碰撞概率从约 0.72 提升到约 0.92。⁶⁷

常见误区

常见误区： B 和 R 的独立调优。许多初学者误以为单纯增加 B 或 R 就能任意改善性能，但实际上二者需要**联合调优**：

- 仅增加 B 而不增加 R ：虽然高相似度对的碰撞概率提高，但低相似度对的误报率也会上升。
- 仅增加 R 而不增加 B ：虽然误报率降低，但真正的重复对也可能因匹配条件过于严格而被漏掉。
- 同时增加 B 和 R ：可以使相变更尖锐，但计算成本（ $K = B \times R$ 次哈希计算）会线性增加。

相变的中心位置大致位于 $J \approx (1/B)^{1/R}$ 。若希望相变中心在 $J = 0.9$ 附近，需要将 B 和 R 调节至满足 $(1/B)^{1/R} \approx 0.9$ 。^a

⁶⁴视频来源时间：00:39:11--00:42:35

⁶⁵视频来源时间：00:42:35--00:43:51

⁶⁶视频来源时间：00:45:04--00:45:51

⁶⁷视频来源时间：00:45:51--00:46:22

^a视频来源时间: 00:46:22--00:47:29

实际论文参数：在一篇大规模去重论文中，作者使用了 $B = 20$ 个波段，每个波段 $R = 450$ 个哈希函数，总计 $K = 9000$ 个 MinHash 签名。在这一配置下，相变中心约为 0.64，即当 $J > 0.64$ 时碰撞概率迅速趋近于 1，当 $J < 0.64$ 时迅速趋近于 0。若目标是过滤 $J > 0.9$ 的文档对，则需要进一步调整参数使相变中心右移至 0.9 附近。⁶⁸

核心要点

MinHash + LSH 的工程优势：

1. **线性时间：**每个文档只需计算 K 个哈希值，无需与其他所有文档两两比较。
2. **可并行化：**MapReduce 框架天然适合此算法——每个节点独立计算签名，再通过哈希桶聚合候选对。
3. **可调节性：**通过 B 和 R 的调节，可以在召回率与精确率之间灵活权衡。

最后需要注意，去重不仅要在单个数据集内部进行，还需要在**多个数据集之间**进行交叉去重。实践中数据集之间常有重复内容，忽略这一步骤会导致训练数据与测试数据之间的污染。⁶⁹

3.4 本章小结

本章系统阐述了数据去重的动机、类型与核心算法：

1. **重复类型：**精确重复（镜像站点、代码克隆）与近似重复（许可证模板、页眉页脚、排版差异）在网络数据中广泛存在，会浪费计算资源并导致记忆与过拟合问题。
2. **精确去重：**基于哈希表的线性时间方法，C4 数据集采用三句话片段的精确匹配策略，但会破坏文档连贯性。
3. **近似去重：**MinHash 利用随机哈希将 Jaccard 相似度估计转化为哈希碰撞概率；LSH 通过 AND-OR 波段结构将碰撞概率曲线塑造为 S 型，实现阈值附近的相变效应。
4. **参数调优：** B （波段数）与 R （每段哈希数）需联合调节以控制相变位置与陡峭程度。实际论文中 $B = 20, R = 450$ 的配置提供了参考基准。
5. **工程实践：**去重操作在物理上位于过滤之后、混合之前，是数据流水线中不可或缺的一环；同时需注意跨数据集的交叉去重与数据净化。

4 数据混合

经过数据转换、过滤与去重后，我们得到了多个精炼且高质量的数据源（domain）。然而，语言模型的预训练并非基于单一来源，而是需要将网页（Common Crawl）、书籍（Books）、代码（Code）、论文（ArXiv）等多种来源组合在一起。如何为这些数据源分配采样权重，即为**数据混合**（Data Mixing）的核心问题。本节时间范围为⁷⁰。

⁶⁸视频来源时间: 00:46:37--00:48:10

⁶⁹视频来源时间: 00:48:55--00:49:23

⁷⁰视频来源时间: 00:49:23--01:13:07

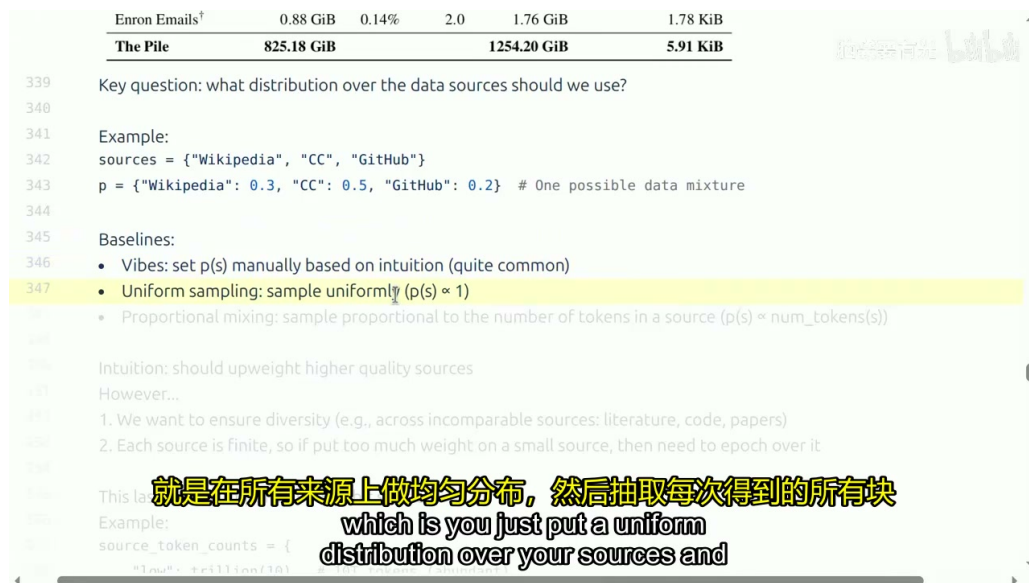


Figure 9: 多数据源的混合权重分配问题

71

4.1 基本问题

在实际操作中，数据混合权重的确定长期依赖启发式与人工经验。早期的做法包括：

- 均匀混合** (Uniform Mixing)：在所有来源上均匀采样，每个来源的权重相等。
- 比例混合** (Proportional Mixing)：按各来源的 token 总量比例采样，数据量越大的来源权重越高。
- 手动启发式** (Manual Heuristic)：基于领域知识手动调整权重，例如提高高质量来源（如 Wikipedia、书籍）的比重。

这些方法的共同缺陷在于缺乏系统性优化。均匀混合可能忽视来源间的质量差异；比例混合则可能让低质量但体量巨大的来源（如原始网页爬取数据）消耗过多训练 token；而手动调整往往依赖直觉，难以推广到数十个来源的复杂场景。⁷²

此外，不同来源之间通常**不可直接比较**。代码与论文、网页与书籍分别对应不同的能力维度，无法简单地说论文一定优于代码。一个通用的语言模型必须保持来源的多样性，不能过度偏向某一类数据。⁷³

4.2 数据源规模与训练轮次的约束

数据混合中一个极易被忽视但至关重要的问题是：每个来源的数据量是有限的。如果为小规模高质量来源分配了过高的采样权重，模型将在训练过程中迅速**耗尽** (exhaust) 该来源的数据，被迫在剩余轮次中重复训练相同内容，导致过拟合。⁷⁴

⁷² 视频来源时间：00:50:38--00:52:04

⁷³ 视频来源时间：00:52:47--00:53:12

⁷⁴ 视频来源时间：00:53:12--00:53:44

核心要点

设第 i 个数据源的 token 总量为 D_i ，混合权重为 λ_i （满足 $\sum_i \lambda_i = 1$ ），总训练 token 量为 N 。则该数据源的实际训练轮次（epoch）为：

$$\text{epochs}_i = \frac{\lambda_i N}{D_i} \quad (1)$$

若 $\text{epochs}_i > 1$ ，意味着该来源的数据被重复使用；若 $\text{epochs}_i \gg 1$ ，则存在严重的重复训练与过拟合风险。

Table 2: 均匀混合下不同数据源的 epoch 计算示例（ $N = 10^{12}$ ）

数据源	Token 总量 D_i	权重 λ_i	实际轮次 epochs_i
低质量网页	10^{13}	0.5	0.05
高质量书籍	10^{10}	0.5	50

上例表明，在均匀混合下，低质量网页仅被使用了 5%，而高质量书籍却被重复训练了 50 轮。这种极端的不平衡在实际训练中是灾难性的：最坏情况下会导致严重的过拟合，最好情况也是计算资源的浪费。⁷⁵

为解决这一问题，研究者提出了**轮次上限**（Epoch Capping）策略。其核心思想是：为每个数据源设定一个最大训练轮次 $E_{\max}$ （例如 20 轮），一旦达到上限，便停止从该来源采样，转而从其他来源补充。形式化约束为：

$$\lambda_i N \leq E_{\max} \cdot D_i \quad (2)$$

该策略最早在多语言模型训练中被系统地提出（如 UniMax 方法），用于处理低资源语言的数据稀缺问题。⁷⁶

4.3 基于回归的混合优化

当面对数十个数据源时，手动调整权重既不现实也不可靠。更严谨的方法是基于**回归的混合优化**（Regression-based Mixing Optimization），代表性工作包括 RegMix 与 DoReMi。⁷⁷

背景知识

代理模型（Proxy Model）是指在回归混合优化中使用的小规模语言模型（通常为数千万到数亿参数）。通过在代理模型上快速训练多种混合比例，收集对应的评估损失，再拟合回归模型预测大规模训练的损失，从而避免直接在大模型上进行昂贵的网格搜索。

基于回归的混合优化流程如下：

1. **采样混合比例**：从某个分布（如狄利克雷分布，Dirichlet Distribution）中采样多组混合权重向量 $\lambda^{(1)}, \lambda^{(2)}, \dots, \lambda^{(K)}$ 。

⁷⁵ 视频来源时间：00:53:44--00:56:03

⁷⁶ 视频来源时间：00:58:33--00:59:43

⁷⁷ 视频来源时间：01:00:03--01:00:38

2. **代理模型训练**: 用每组混合权重训练小规模代理模型, 记录其在下游任务或困惑度 (Perplexity) 上的损失 $L^{(1)}, L^{(2)}, \dots, L^{(K)}$ 。
3. **回归拟合**: 将 $(\lambda^{(k)}, L^{(k)})$ 作为训练数据, 拟合一个回归模型 $\hat{L} = f(\lambda)$ 。常用的回归方法包括线性模型、梯度提升决策树 (GBDT) 以及对数线性模型 (Log-linear Model)。
4. **优化求解**: 在回归模型上求解最优混合权重:

$$\lambda^* = \arg \min_{\lambda} f(\lambda) \quad \text{s.t.} \quad \sum_i \lambda_i = 1, \lambda_i \geq 0 \quad (3)$$

5. **大规模训练**: 将优化后的 λ^* 应用于大规模模型的实际训练。

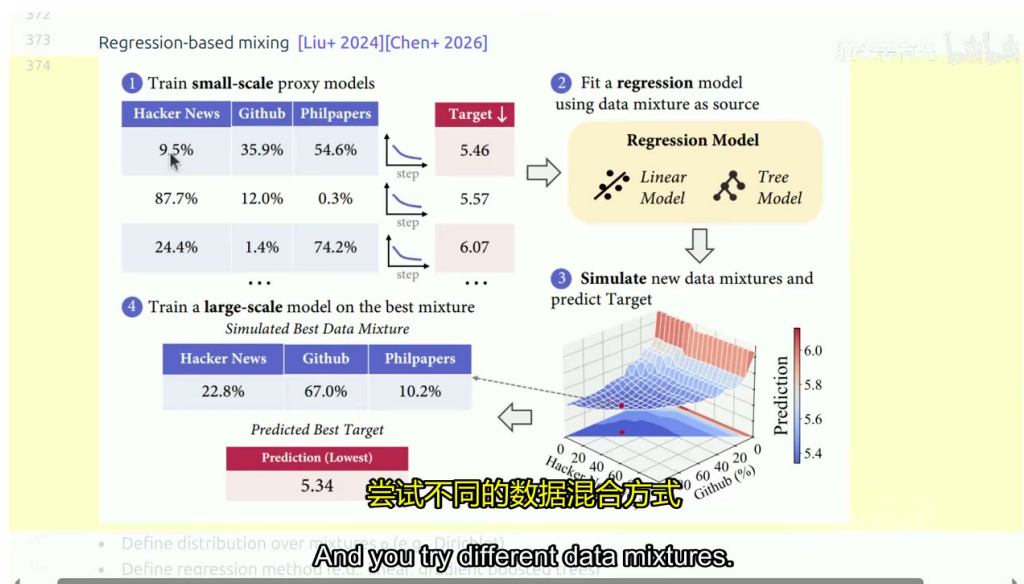


Figure 10: RegMix / DoReMi 的回归优化流程

78

该方法与规模扩展定律 (Scaling Laws) 的思想一脉相承: 通过廉价的代理实验推断昂贵的大规模行为。⁷⁹

常见误区

回归优化的目标函数通常基于**下游评估指标** (Downstream Evaluation Metrics)。然而, 预训练的目标是培养通用能力, 而非拟合特定的下游任务。如果评估任务中代码类任务占比过高, 优化过程会赋予代码数据过高的权重, 导致模型在诗歌生成等其他任务上表现退化。这种**过拟合评估指标**的风险必须警惕。相比之下, 均匀混合与比例混合不涉及下游评估, 因此天然避免了这一问题。^a

^a视频来源时间: 01:02:44--01:03:35

此外, 回归模型在优化时存在两个关键假设需要谨慎对待:

⁷⁹视频来源时间: 01:00:38--01:02:18

- **泛化性假设**：回归模型在训练数据（随机采样的混合比例）上拟合良好，但优化时可能走向极端区域，导致外推误差增大。
- **规模不变假设**：最优混合比例从小规模代理模型直接推广到大规模模型。然而，规模效应（Scaling Effect）意味着最优混合并非恒定——随着训练 token 量的增加，低质量数据的边际效用可能上升，高质量数据的边际效用则可能下降。⁸⁰

4.4 从小规模到大规模的扩展

代理模型实验与大规模训练之间存在显著的规模差异。直接从小规模最优混合推广到大模型，可能因数据源耗尽而产生误导。为解决这一问题，研究者提出了**模拟轮次**（Simulated Epochs）技术。⁸¹

模拟轮次的核心思想是：让小规模实验**模仿**大规模实验的数据稀缺情况。具体做法是按比例缩小（下采样，Downsampling）各数据源的规模，使得代理模型在少量 token 上的训练体验与大模型在大量 token 上的训练体验一致。

例如，若大规模训练使用 $N_{\text{large}} = 10^{15}$ 个 token，而代理实验仅使用 $N_{\text{small}} = 10^{13}$ 个 token，则下采样比例为：

$$r = \frac{N_{\text{small}}}{N_{\text{large}}} = 0.01 \quad (4)$$

对每个数据源按比例 r 进行下采样后，代理模型在有限数据上的 epoch 数将与大模型相当。这样做的效果是：优化过程会意识到小规模高质量来源（如 Wikipedia）在放大后将被严重过度训练，从而主动寻找更平衡的混合方案。⁸²

下采样也存在风险：若某来源在按比例缩小后 token 数过少，可能导致代理模型无法有效学习该领域的表示，甚至因舍入误差被分配零权重。一种缓解策略是设定最低训练轮次（例如至少训练 1 轮），确保每个来源都被充分采样。⁸³

4.5 领域细分与混合粒度

数据混合不仅可以应用于不同来源（如网页、书籍、代码），还可以在更细的粒度上进行。例如，将 Common Crawl 的网页数据按 URL 分类器划分为不同主题领域（如新闻、学术、娱乐），或按质量评级划分为多个等级。这样便得到一个**领域 × 质量**的二维网格，每个单元格都可以作为独立的混合组分。⁸⁴

这种细粒度混合的优势在于：

- 能够自动发现最优的领域划分，无需人工预设来源边界；
- 可以在同一数据集内实现质量与多样性的联合优化；
- 为回归优化提供更丰富的输入维度，提升拟合精度。

⁸⁰视频来源时间：01:04:50--01:06:27

⁸¹视频来源时间：01:06:27--01:07:51

⁸²视频来源时间：01:07:51--01:09:54

⁸³视频来源时间：01:10:43--01:11:38

⁸⁴视频来源时间：01:11:47--01:12:43

4.6 训练中的批次级实现

在实际训练中，数据混合并非以单个 token 为单位进行采样，而是以**序列**（sequence）为单位。具体而言，每个训练批次（batch）由多个完整的文本序列组成，这些序列是从当前的数据混合分布中采样得到的。⁸⁵

这里有一个重要的工程细节：为了降低采样方差，混合通常是在**多个批次之间**实现的，而不是在单个批次内强制按比例分配。例如，如果某来源的混合权重为 10%，并不意味着每个批次都必须恰好包含 10% 来自该来源的序列——这样做会导致单个批次内的样本高度受限。相反，系统会在连续多个批次中维持大致的比例，允许单个批次有所波动，从而在统计意义上逼近目标混合比例，同时保持每个批次内部的多样性。

这种批次级实现还意味着：当某一来源的数据被耗尽（epoch 超过上限）时，系统需要重新从剩余来源中采样序列来填充批次。如果未做好 epoch 约束与批次填充策略的协同设计，可能出现“空洞批次”——即某些批次因数据来源不足而被迫重复采样已见过的内容，从而加剧过拟合风险。

4.7 本章小结

数据混合是预训练数据处理的最后一步，决定了模型从哪些来源、以何种比例获取知识。本章的核心要点包括：

- 简单的均匀混合与比例混合缺乏系统性，且容易忽视数据源规模差异带来的 epoch 失衡问题。
- 实际训练轮次由 $\text{epochs}_i = \lambda_i N / D_i$ 决定，小规模高质量来源在高权重下极易被过度训练。
- **轮次上限**（Epoch Capping）是一种直接有效的约束策略，可防止数据源耗尽。
- **基于回归的混合优化**（RegMix / DoReMi）通过代理模型实验拟合回归模型，以低成本搜索最优混合比例，但需警惕过拟合评估指标与规模效应带来的外推误差。
- **模拟轮次**（Simulated Epochs）通过下采样使小规模实验模仿大规模数据稀缺情况，是连接代理模型与大规模训练的关键桥梁。
- 混合粒度可以从粗粒度的来源级别细化到领域 $\times$ 质量的二维网格，进一步提升优化的灵活性与精度。

数据混合的本质是一个多目标优化问题：在多样性、质量、数据稀缺性与计算成本之间寻求平衡。任何优化都必须谨慎，否则可能优化了错误的指标，导致模型在通用能力上的退化。

5 后训练数据与合成数据

前几节讨论的转换、过滤、去重与混合，均服务于预训练阶段——其目标是利用大规模无标注数据培养语言模型的基本能力。从本节开始，我们将进入后训练（post-training）阶段，数据范式发生根本转变：从“通用 + 无标注”转向“任务特定 + 合成标注”。

⁸⁵ 视频来源时间：00:57:03--00:58:14

核心要点

后训练数据的核心范式：定义环境/任务 → 教师模型（Teacher Model）生成答案 → 筛选与验证 → 构建数据集。在开放社区中，几乎所有后训练数据都是生成的（synthetic），而非人工采集。

5.1 后训练数据概述

后训练阶段的数据与预训练数据存在本质区别。预训练数据相对任务无关，旨在培养模型的基本语言理解与生成能力；而后训练数据则高度依赖具体任务，例如数学推理、代码生成、软件工程（SWE）等。⁸⁶

构建后训练数据的标准流程如下：

1. **定义环境与任务：**明确模型需要掌握的能力领域，如 GitHub 仓库中的代码修改、数学竞赛题求解等。
2. **教师模型提供反馈：**使用一个强大的模型（教师模型，Teacher Model）对任务生成答案或解决方案。
3. **筛选与验证：**对生成的答案进行过滤，保留高质量样本；理想情况下通过执行反馈（Execution Feedback）验证正确性。
4. **构建数据集：**将筛选后的（问题，答案）对组织成训练数据集，用于对基础模型进行微调。

背景知识

教师模型（Teacher Model）指用于生成答案、标签或反馈的强大模型。在后训练数据的构建中，教师模型通常是一个规模远大于目标学生模型的先进模型（如 GPT-4 或 DeepSeek-R1）。教师模型提供答案的标准流程已成为开放社区生成后训练数据的主流方法。人工标注虽然质量高，但耗时且成本高昂；因此，除非处于技术前沿且预算充足，大多数团队选择用模型生成替代或辅助人工标注。^a

^a视频来源时间：01:13:52--01:14:52

⁸⁶视频来源时间：01:13:07--01:13:33

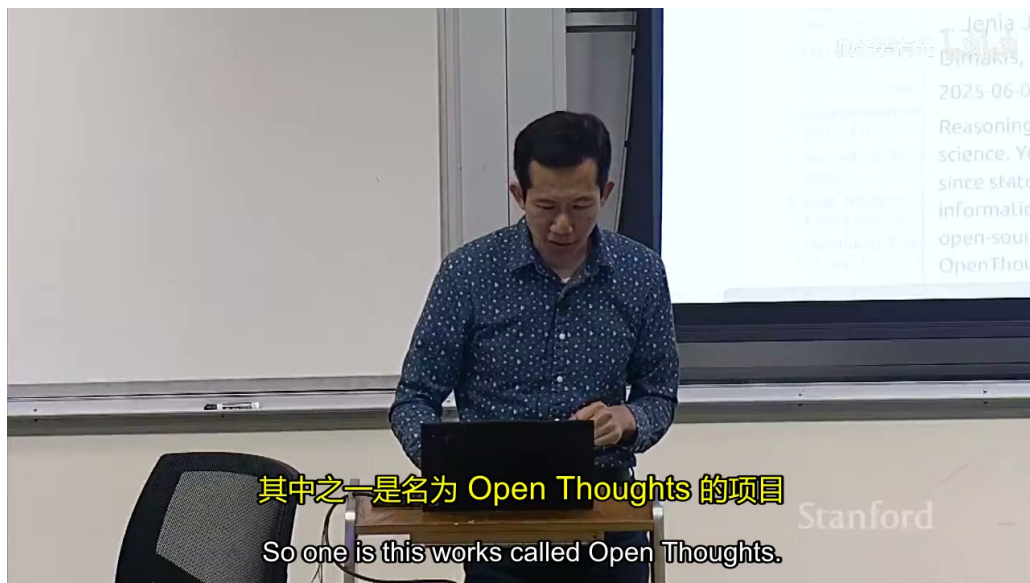


Figure 11: 后训练数据构建的通用范式

87

5.2 数学与科学推理合成数据

数学与科学推理是后训练数据应用最成熟的领域之一。OpenMathInstruct 项目是该方向的代表性工作，其目标是构建大规模的数学与科学推理训练数据集。⁸⁸

OpenMathInstruct 的生成流程包含以下关键步骤：

1. **多来源收集问题：**从多种来源获取数学与科学问题，包括人工编写的数据集（如 GSM8K、MATH）以及合成来源。
2. **采样多个生成结果：**对每个问题，使用教师模型采样多个答案（例如 16 个）。研究表明，采样多个生成结果并筛选，比单一采样效果更好。⁸⁹
3. **答案筛选与去重：**对生成的答案进行筛选，去除重复和高质量样本。
4. **构建最终数据集：**最终形成包含约 120 万个案例的数据集。需要注意的是，120 万是（问题，答案）对的数量；由于每个问题采样了约 16 个答案，真实独立问题的数量约为 120 万除以 16。⁹⁰

在教师模型的选择上，OpenMathInstruct 团队发现了一个反直觉的现象：

常见误区

教师模型悖论：更好的模型并不一定更适合当教师。OpenMathInstruct 的实验表明，Qwen2.5-32B（一个相对较小且较旧的模型）作为教师时，效果反而优于当时最强大的开源模型之一 DeepSeek-R1。这说明教师模型的选择并非简单地挑选最强模型，而是需要综合考虑模型生成答案的可学习性、多样性以及与目标学生模型的匹配程度。^a

^a视频来源时间：01:16:47--01:17:10

⁸⁸视频来源时间：01:14:56--01:15:27

⁸⁹视频来源时间：01:16:41--01:16:55

⁹⁰视频来源时间：01:17:17--01:17:42

此外，研究还发现基本的答案筛选（例如只保留正确格式的答案）对最终效果帮助有限，关键在于多采样和合理的去重策略。⁹¹

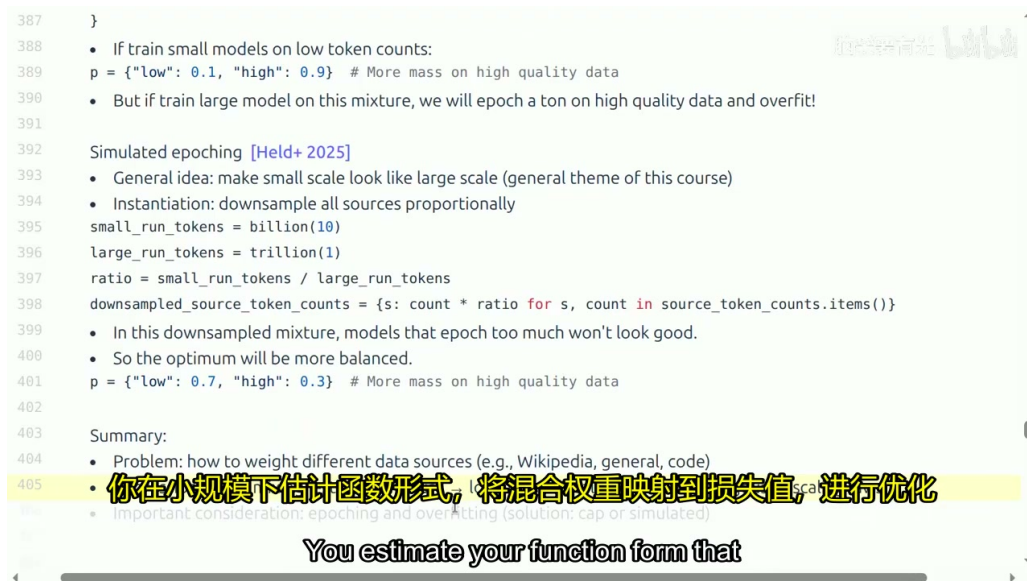


Figure 12: 合成数据项目概览（OpenMathInstruct、SWE-smith 等）

92

图

5.3 代码与智能体合成数据

除了数学推理，代码生成与智能体（Agent）能力是后训练数据的另一大热点领域。近年来，研究者们不仅关注模型能否生成代码片段，更关注模型能否完成完整的软件开发任务。⁹³

5.3.1 SWE-smith：从代码仓库自动生成任务

SWE-smith 提出了一种从代码仓库自动生成训练任务的方法。其核心思想是：给定一个 GitHub 仓库，通过语言模型自动生成代码修改任务。生成的任务通常涉及对代码的修改，这些修改可能引入错误；经过验证后，形成可用于训练的任务实例。⁹⁴

SWE-smith 在当时生成了约 5 万个任务，这一规模在发布时已经相当大。此后，该领域迅速发展，出现了许多后续工作。

5.3.2 Agentless / OpenHands：智能体轨迹生成

NVIDIA 发表的 Agentless 论文提出了一个有趣的观点：与数学领域不同，软件工程任务（SWE-bench 等）通常有大量依赖项，大多数 GitHub 仓库无法直接运行——需要安装各种可能已过时的依赖，特别是回退到某个混乱的 pull request 时，环境配置简直是噩梦。⁹⁵

因此，Agentless 团队探索了无执行反馈的方案：他们发现模型足够强大，可以在没有执行反馈的情况下解决许多任务。实验结果显示：

⁹¹ 视频来源时间：01:17:10--01:17:17

⁹³ 视频来源时间：01:17:53--01:18:06

⁹⁴ 视频来源时间：01:18:07--01:18:55

⁹⁵ 视频来源时间：01:19:03--01:19:38

- 允许执行时，准确率约为 80%；
- 禁止执行时，准确率仍接近 70%。⁹⁶

这表明现代模型本身已经具备了较强的代码语义理解能力，不依赖实际执行也能取得不错的效果。

OpenHands 框架则采用了不同的路径：他们从大型模型中提炼智能体轨迹，并进行了严格的过滤（例如防止智能体被“黑客攻击”、防止模型忽略指令强行执行代码）。OpenHands 3.0 明确规定智能体不能运行 Python 代码，仅限执行 `git status`、`grep` 等基础操作。他们成功生成了约 30 万个智能体轨迹，这些轨迹比 SWE-smith 的合成样本更加逼真，因为它们来自真实的 GitHub pull request。⁹⁷

5.3.3 SWE-bench：大规模拉取请求数据集

SWE-bench 是一个尝试获取大量真实 pull request 的数据集。研究者获取了大量 GitHub 仓库，尝试安装并运行测试；大部分尝试会失败，但他们会不断尝试，然后通过语言模型生成修复方案。⁹⁸

最新发布的 SWE-bench 扩展工作可以从 SWE-smith 的方法开始，扩展至 1200 万个智能体轨迹。Agentless 3.0 的优势在于轻量级：它使用了 SWE-bench 任务，但仅成功执行了其中约 3.2 万个（120 个未能执行）；由于 Agentless 3.0 对执行不敏感，这些未执行的样本也可以全部用于训练。⁹⁹

5.4 合成数据的权衡

合成数据的构建涉及多个关键权衡：

1. 完全合成 vs 半合成：

- **完全合成：**任务和答案均由模型生成，可控性强，但可能缺乏真实世界的复杂性。
- **半合成：**保持真实环境（如真实 GitHub 仓库），仅合成任务或答案。这种方法通常更贴近实际应用场景，但环境配置和依赖管理非常繁琐。¹⁰⁰

2. 数据规模增长趋势：

后训练数据集的规模正在快速增长。从最初数万级别的 SWE-smith，到 30 万轨迹的 OpenHands，再到 1200 万轨迹的最新扩展，数据集规模呈指数级增长。¹⁰¹

3. 质量筛选与执行反馈：

合成数据并非越多越好。OpenMathInstruct 发现较少的来源效果更好，而不是试图使用所有来源；Agentless 则展示了无执行反馈时仍可达到接近有执行反馈的效果。这些发现提示我们，合成数据的质量筛选策略与教师模型的选择同样重要。

⁹⁶视频来源时间：01:20:05--01:20:18

⁹⁷视频来源时间：01:20:24--01:21:38

⁹⁸视频来源时间：01:21:45--01:22:11

⁹⁹视频来源时间：01:22:21--01:22:54

¹⁰⁰视频来源时间：01:23:21--01:23:40

¹⁰¹视频来源时间：01:22:54--01:23:13

Table 3: 完全合成与半合成数据对比

维度	完全合成	半合成（真实环境 + 合成任务）
环境真实性	低	高
可控性	高	中
依赖管理复杂度	低	高
数据规模扩展性	高	中
典型代表	SWE-smith	OpenHands、SWE-bench

5.5 本章小结

本节从预训练数据转向了后训练数据，重点讨论了合成数据（Synthetic Data）在数学推理、代码生成与智能体能力培养中的应用。

- **范式转变**：后训练数据从”通用无标注”转向”任务特定 + 合成标注”，教师模型提供反馈是核心机制。
- **数学推理**：OpenMathInstruct 通过多来源问题 + 多答案采样 + 筛选，构建了 120 万案例的数据集；但**更好的模型不等于更好的教师**。
- **代码与智能体**：SWE-smith 开创了从代码仓库自动生成任务的先河；Agentless 证明无执行反馈也能取得不错效果；OpenHands 通过真实 GitHub pull request 生成了 30 万高质量智能体轨迹；SWE-bench 及其扩展将规模推至 1200 万。
- **关键权衡**：完全合成 vs 半合成、数据规模 vs 质量筛选、执行反馈 vs 模型语义理解能力，都是构建后训练数据时必须仔细权衡的问题。

正如讲师所强调的，数据工作实际上非常复杂繁琐，需要具体领域知识并通过具体实例来构建高质量数据集。本节仅是对后训练数据与合成数据这一 rapidly evolving 领域的概览，更多细节需要结合具体项目深入研究。¹⁰²

6 总结与延伸

6.1 核心要点回顾

本章围绕语言模型训练中的数据问题，系统性地讲解了从原始网络数据到可用于训练的高质量数据集的完整处理流水线。这一流程可以概括为五个关键环节：

1. **数据转换**（Data Transformation）：将 HTML、PDF 等原始格式转换为可供模型训练的纯文本。核心挑战在于内容边界的判定与复杂结构（如嵌套表格）的线性化。
2. **数据过滤**（Data Filtering）：基于目标高质量数据训练轻量级分类器，从海量原始数据中筛选出符合条件的子集。关键洞察是：过滤阈值与训练规模之间存在非平凡的权衡关系——高质量数据在短训练中优势明显，但长训练下低质量数据的边际效用不可忽略。

¹⁰²视频来源时间：01:23:44--01:24:38

3. **数据去重** (Deduplication): 通过精确匹配与近似匹配 (MinHash + LSH) 消除重复内容, 避免模型记忆与计算资源浪费。MinHash 以线性时间估计 Jaccard 相似度, LSH 通过 AND-OR 波段结构将碰撞概率转化为可调的 S 型曲线。
4. **数据混合** (Data Mixing): 为多个数据源分配采样权重, 并考虑各来源的数据量限制与实际训练轮次。基于回归的优化方法 (RegMix / DoReMi) 通过小规模代理模型实验来预测最优混合比例。
5. **后训练数据与合成数据** (Post-Training & Synthetic Data): 从通用预训练转向任务特定数据, 利用教师模型生成合成训练样本。合成数据已成为代码、数学与科学推理领域后训练的主流范式。

6.2 从数据到能力的因果链条

这五个环节并非孤立的技术模块, 而是构成了一条完整的因果链条:

原始网络数据 $\xrightarrow{\text{转换}}$ 纯文本 $\xrightarrow{\text{过滤}}$ 高质量文本 $\xrightarrow{\text{去重}}$ 非冗余高质量文本 $\xrightarrow{\text{混合}}$ 均衡训练语料
 $\xrightarrow{\text{预训练}}$ 基础模型 $\xrightarrow{\text{后训练}}$ 任务专用模型

每一步的决策都会向下游传递影响。转换阶段的工具选择决定了可处理的格式范围; 过滤标准的主观性可能引入隐含的偏见; 去重参数的过严或过松直接影响数据多样性; 混合比例的失衡则可能导致某些能力维度的系统性缺失。

6.3 开放问题与前沿方向

背景知识

1. **合成数据的质量天花板**: 当前合成数据高度依赖教师模型的能力。如果教师模型本身存在系统性错误, 这些错误将被传播到学生模型的训练数据中。如何打破这一“教师模型悖论”, 是一个尚未解决的核心问题。
2. **数据混合的规模外推**: 代理模型实验得出的最优混合比例是否适用于更大规模的模型? 现有证据表明规模效应显著, 但系统性的外推理论仍然缺失。
3. **多模态数据的扩展**: 本章讨论主要围绕文本数据, 但当前前沿模型已广泛涉及图像、音频、视频等多模态数据。如何将转换、过滤、去重与混合的方法论扩展到多模态场景, 是一个活跃的研究方向。
4. **数据治理与伦理**: 随着数据规模从万亿级向十万亿级迈进, 数据来源的合法性、版权合规性以及隐私保护问题将变得愈发尖锐。技术解决方案必须与法律框架协同演进。

6.4 实践建议

对于希望构建高质量语言模型的实践者, 以下原则值得遵循:

常见误区

1. **不要低估数据工程：**数据处理的投入往往与模型架构的创新同等重要。一个精心设计的数据流水线，其效果可能远超一次结构性的模型改动。
2. **建立可复现的数据基线：**在尝试复杂的过滤或混合策略之前，先建立简单但可靠的基线（如比例混合 + 基本过滤），以便后续改进有明确的参照。
3. **监控下游评估指标：**数据质量的判定最终取决于模型在下游任务上的表现，而非中间环节的统计指标。避免过度优化易于计算但缺乏预测力的代理指标。
4. **保留原始数据与处理日志：**数据决策具有高度的经验性和迭代性。保留完整的处理链路和版本历史，是后续复盘与改进的基础。

数据来源: *Stanford CS336 Language Modeling from Scratch, Spring 2026*

Bilibili BV12gLx6xEcQ P14 / 笔记生成日期: 2026 年 6 月 8 日